Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский
политехнический университет»**

Электротехнический факультет
Кафедра «Информационные технологии и автоматизированные системы»
направление подготовки: 09.03.01– «Информатика и вычислительная техника»

Лабораторная работа № 1
по дисциплине
«Информатика»
на тему
«Знакомство с C++. Выполнение программы простой структуры»
Вариант 12

Выполнил студент гр. ИКС-26-1б
Захлевных Иван Алексеевич

Проверил:
Духанин Алексей Владимирович

(оценка) (подпись)

(дата)

г. Пермь, 2026

ВВЕДЕНИЕ

Вариант: 12

Цель: для задания 1 - вычислить значение заданного выражения с использованием различных вещественных типов данных. Для задания 2 - вычислить значения выражений и объяснить результаты.

Задачи:

1. Написать программу решения задачи 1 в соответствии с требованиями.
2. Сравнить результаты выполнения для различных вещественных типов данных.
3. Объяснить полученные для задачи 1 результаты.
4. Написать программу решения задачи 1 в соответствии с требованиями.
5. Объяснить полученные для задачи 2 результаты.

1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

С поставленными задачами приведены следующие методические указания:

- для ввода и вывода данных использовать операции >> и << и стандартные потоки cin и cout;

- для вычисления степени можно использовать функцию pow(x,y) из библиотечного файла math.h;

- при выполнении задания 1 надо использовать вспомогательные переменные для хранения промежуточных результатов.

Эти указания, в первую очередь, ограничивают возможности использования альтернативных утилит ввода/вывода и обязывают использование вспомогательных переменных для задания 1.

2 ХОД ВЫПОЛНЕНИЯ РАБОТЫ

2.1 Задание 1

В представленном варианте дано следующее выражение:

$$\frac{(a+b)^2-(a^2+2\,\mathrm{ab})}{b^2}$$

где a = 1000;

b = 0.0001.

Для удобных вычислений с различными типами данных решено использовать template-функцию, ограниченную концептом `std::floating_point`, а также, в силу известности изначальных констант, помеченную `consteval`. Результаты этой функции сохраняются в переменные со спецификатором `constexpr` для гарантии статического выполнения, но не `static` для улучшения локальности кеша (как минимум для amd64) путём замены чтения `.rodata` на извлечение константы напрямую из инструкции, уже находящейся в L1i кеше. Для вычисления квадратов выражений внутри функции была написана отдельная вспомогательная функция, ограниченная на `std::is_arithmetic_v<T>` и обходящая `std::pow` в возможностях оптимизации, `constexpr`-исполнении и наихудшей возможной эффективности. Весь исходный код представлен в приложении А. На рисунке 1 представлена блок-схема функции, отвечающей за решение задания 1.

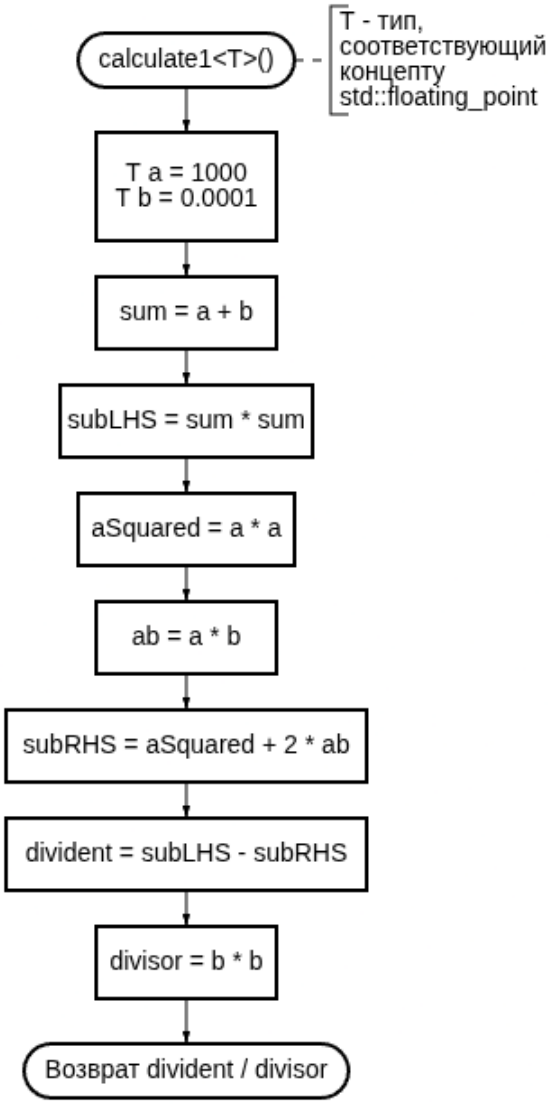


Рисунок 1 - блок-схема функции calculate1

По результатам исполнения кода на архитектуре amd64 (x64) с System V ABI после компиляции с помощью Clang v22.1.8 и флагом `-march=native`, вывод результата типа `float` показал значение сильно отклоняющееся от результата типа `double` (результаты представлены в приложении Б). Оба результата показали несоответствие истинному результату выражения (который равен 1, так как $(a + b)^2 = a^2 + 2ab + b^2$, следовательно, финальное выражение равно $b^2/b^2 = 1$), однако результат типа `double` оказался наиболее к нему приближен с маргинальным отклонением.

Причина данных результатов заключается в точности мантиссы типа `float` в стандартном 32-битном представлении IEEE-754. При многократных операциях на сильно больших и сильно маленьких числах накапливается погрешность, заметно влияющая на финальный результат. В случае с 64-битным `double` эта погрешность имеет намного меньшую неточность, выводя очень близкий ответ.

2.2 Задание 2

Требуется вычислить значения следующих выражений и объяснить полученные результаты:

1. `- -m-++n`
2. `m*n<n++`
3. `n-- > m++`

Зная правила приоритета и обработки операторов C++, можно предположить, что в первом случае выражение будет идентично `-(-m) - (++n)`, или же `m - (++n)`; во втором - `m * n < (n++)`; в третьем - `(n--) > (m++)`. Следовательно, исходя из правил пост- и пре-инкремента/декремента, первое выражение будет идентично `m - n - 1`; второе - `m * n < n`, или `m * n - n < 0`, или `n * (m - 1) < 0`; третье - `n > m`.

В коде функции, вычисляющие результаты выражений берут `n` и `m` по значению, чтобы избежать их модификации инкрементами/декрементами. Код задания 2 также представлен в приложении А.

Сравнивая значения, полученные в результате выполнения функций с предположенными формулами можно заметить 100%-е сходство (приложение Б).

ЗАКЛЮЧЕНИЕ

В результате выполнения работы были исследованы принципы исполнения кода C++ и особенности базовых типов и операторов. Все задачи были выполнены, полученные результаты объяснены (для задания 1 отдельно, для задания 2 в процессе анализа операторов), цели достигнуты.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 7.32-2017. Межгосударственный стандарт. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления

ПРИЛОЖЕНИЕ А

Исходный код

```
#include <iostream>



template <typename T>
     requires std::is_arithmetic_v<T>
[[nodiscard]] constexpr T square(T x) noexcept {
     return x * x;
}

template <std::floating_point T>
[[nodiscard]] consteval T calculate1() noexcept {
     T a = 1000.;
     T b = 0.0001;

     auto sum = a + b;
     auto subLHS = square(sum);

     auto aSquared = square(a);
     auto ab = a * b;
     auto subRHS = aSquared + 2 * ab;

     auto divident = subLHS - subRHS;
     auto divisor = square(b);

     return divident / divisor;
}

void task1() {
     std::cout << "Task 1\n";

     constexpr auto floatRes = calculate1<float>();
     constexpr auto doubleRes = calculate1<double>();

     std::cout << "float: " << floatRes << "\ndouble: " <<
doubleRes << "\n\n";
```

```
    return;
}



[[nodiscard]] std::int32_t calculate21(std::int32_t n,
std::int32_t m) noexcept {
    return - -m-++n; // -(-m) - (++n)   |   m - (++n)   |   m - n
- 1
}


[[nodiscard]] std::int32_t calculate22(std::int32_t n,
std::int32_t m) noexcept {
    #pragma GCC diagnostic push
    #pragma GCC diagnostic ignored "-Wunsequenced"


    // NOLINTNEXTLINE(bugprone-inc-dec-in-conditions)
    return m*n<n++; // (m * n) < (n++)   |   m * n < n   |   m *
n - n < 0   |   n * (m - 1) < 0


    #pragma GCC diagnostic pop
}


[[nodiscard]] std::int32_t calculate23(std::int32_t n,
std::int32_t m) noexcept {
    return n-- > m++; // (n--) > (m++)   |   n > m
}


template <bool Last = false>
void calculate2(std::int32_t n, std::int32_t m) {
    std::cout << "n = " << n << "; m = " << m << "\n";


    std::cout << std::boolalpha <<
        "  1. " << calculate21(n, m) << "\n"
        "  2. " << static_cast<bool>(calculate22(n, m)) << "\n"
        "  3. " << static_cast<bool>(calculate23(n, m)) << "\n"
        "Assumed:\n"
```

```
            "  1. " << m - n - 1 << " (m - n - 1)\n"
            "  2. " << (n * (m - 1) < 0) << " (n * (m - 1))\n"
            "  3. " << (n > m) << " (n > m)\n";

    if constexpr (!Last)
        std::cout << "\n";

    return;
}

void task2() {
    std::cout << "Task 2\n";

    calculate2(5, 3);
    calculate2(-8, 4);
    calculate2(0, 6);
    calculate2(0, 0);
    calculate2(45, -12);
    calculate2(1, 1);
    calculate2(1, 0);
    calculate2<true>(0, 1);

    return;
}


int main() {
    task1();
    task2();

    return 0;
}
```

ПРИЛОЖЕНИЕ Б

Вывод программы

```
Task 1
float: 6.25e+06
double: 1.00117

Task 2
n = 5; m = 3
  1. -3
  2. false
  3. true
Assumed:
  1. -3 (m - n - 1)
  2. false (n * (m - 1))
  3. true (n > m)

n = -8; m = 4
  1. 11
  2. true
  3. false
Assumed:
  1. 11 (m - n - 1)
  2. true (n * (m - 1))
  3. false (n > m)

n = 0; m = 6
  1. 5
  2. false
  3. false
Assumed:
  1. 5 (m - n - 1)
  2. false (n * (m - 1))
  3. false (n > m)

n = 0; m = 0
  1. -1
  2. false
```

```
  3. false
Assumed:
  1. -1 (m - n - 1)
  2. false (n * (m - 1))
  3. false (n > m)

n = 45; m = -12
  1. -58
  2. true
  3. true
Assumed:
  1. -58 (m - n - 1)
  2. true (n * (m - 1))
  3. true (n > m)

n = 1; m = 1
  1. -1
  2. false
  3. false
Assumed:
  1. -1 (m - n - 1)
  2. false (n * (m - 1))
  3. false (n > m)

n = 1; m = 0
  1. -2
  2. true
  3. true
Assumed:
  1. -2 (m - n - 1)
  2. true (n * (m - 1))
  3. true (n > m)

n = 0; m = 1
  1. 0
  2. false
```

```
  3. false
Assumed:
  1. 0 (m - n - 1)
  2. false (n * (m - 1))
  3. false (n > m)
```